

Jour 2 : GitLab Runners et Executors - Optimisation des Pipelines

Programme de la journee

Horaire	Sujet
09h00 - 10h30	Runners GitLab : concepts et installation
10h30 - 10h45	Pause
10h45 - 12h15	Executors et Docker-in-Docker
12h15 - 13h30	Pause dejeuner
13h30 - 15h00	Cache, Artifacts et optimisation
15h00 - 15h15	Pause
15h15 - 16h45	Variables CI/CD et securite
16h45 - 17h00	Recapitulatif et Q&A

Partie 1 : Qu'est-ce qu'un Runner GitLab ?

L'analogie de la construction

Imaginez que GitLab CI/CD est un **chef de chantier**. Il lit les plans (le fichier `.gitlab-ci.yml`) et sait exactement ce qu'il faut faire. Mais il ne fait pas le travail lui-meme !

Il a besoin d'**ouvriers** pour executer les taches. Ces ouvriers, ce sont les **Runners**.

```
Vous (developpeur)
  |
  v
GitLab (chef de chantier) --- lit le .gitlab-ci.yml (les plans)
  |
  v
Runner (ouvrier) --- execute les commandes (le travail)
  |
  v
Resultat (batiment construit) --- votre application testee/deployee
```

Definition formelle

Un **GitLab Runner** est un agent (un programme) qui s'exécute sur une machine (votre ordinateur, un serveur, le cloud) et qui attend les instructions de GitLab.

Quand vous faites un **git push**, GitLab regarde votre **.gitlab-ci.yml** et envoie les tâches (jobs) au Runner disponible. Le Runner exécute les commandes et renvoie le résultat à GitLab.

Le cycle de vie d'un job

```
1. Vous faites "git push"
2. GitLab detecte le push et lit .gitlab-ci.yml
3. GitLab cree des "jobs" (taches a executer)
4. GitLab cherche un Runner disponible et compatible
5. Le Runner telecharge votre code (git clone/fetch)
6. Le Runner execute les commandes du job (script)
7. Le Runner envoie les logs en temps reel a GitLab
8. Le Runner envoie le resultat (succes/echec) a GitLab
9. Le Runner nettoie son environnement
10. Le Runner attend le prochain job
```

Partie 2 : Types de Runners

Il existe **trois types** de Runners dans GitLab. Chacun a un perimetre different.

Tableau comparatif

Caracteristique	Shared Runner	Group Runner	Specific Runner
Perimetre	Tous les projets	Un groupe de projets	Un seul projet
Qui l'installe	Admin GitLab	Admin du groupe	Mainteneur du projet
Partage	Tout le monde	Membres du groupe	Equipe du projet
Cas d'usage	Taches generiques	Equipe/departement	Besoin specifique
File d'attente	Potentiellement longue	Moderee	Courte
Personnalisation	Limitee	Moyenne	Totale
Cout	Mutualise	Par groupe	Dedie

Shared Runner (Runner partage)

C'est un Runner mis a disposition par l'administrateur GitLab pour **tous les projets**.

Avantages :

- Pas besoin d'installer quoi que ce soit

- Maintenance centralisée
- Idéal pour débuter

Inconvénients :

- Temps d'attente si beaucoup de projets l'utilisent
- Configuration générique (pas optimisée pour votre projet)
- Risques de sécurité (environnement partagé)

Exemple concret : Sur gitlab.com, les Shared Runners sont des machines dans le cloud Google. Quand vous créez un projet sur gitlab.com, vous pouvez immédiatement lancer des pipelines grâce à ces Runners partagés.

Group Runner (Runner de groupe)

Un Runner attribué à un **groupe GitLab** (et tous ses sous-projets).

Exemple concret : Votre entreprise a un groupe "équipe-backend" avec 10 projets.

Un Group Runner est installé sur un serveur dédié à cette équipe. Seuls les 10 projets de ce groupe peuvent l'utiliser.

Specific Runner (Runner spécifique)

Un Runner attribué à **un seul projet**.

Exemple concret : Votre projet a besoin d'un GPU pour entraîner un modèle d'IA.

Vous installez un Runner sur une machine avec GPU et l'assignez uniquement à ce projet.

Partie 3 : Installation d'un Runner pas à pas

Prérequis

- Une machine (votre PC, un serveur, une VM)
- Docker installé (recommandé)
- Accès à votre instance GitLab

Étape 1 : Télécharger le Runner

```
# ===== METHODE 1 : Installation avec Docker (RECOMMANDEE) =====

# On telecharge l'image officielle du Runner GitLab
# Cette image contient tout le necessaire pour executer un Runner
docker pull gitlab/gitlab-runner:latest

# ===== METHODE 2 : Installation directe sur Linux =====

# On ajoute le depot officiel de GitLab
curl -L "https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh" | sudo bash

# On installe le paquet gitlab-runner
sudo apt-get install gitlab-runner

# ===== METHODE 3 : Installation sur Windows =====

# Telechargez le binaire depuis :
# https://gitlab-runner-downloads.s3.amazonaws.com/latest/binaries/gitlab-runner-windows-amd64.exe
# Renommez-le en gitlab-runner.exe
# Placez-le dans un dossier de votre PATH
```

Etape 2 : Enregistrer le Runner aupres de GitLab

Pour que GitLab sache que votre Runner existe, il faut l'**enregistrer**.

```
# Commande d'enregistrement interactive
# GitLab va vous poser plusieurs questions
sudo gitlab-runner register

# Question 1 : URL de votre instance GitLab
# Exemple : https://gitlab.com/ ou https://gitlab.votre-entreprise.com/
# C'est l'adresse de votre serveur GitLab

# Question 2 : Token d'enregistrement
# Ou le trouver ?
#   - Shared Runner : Admin > CI/CD > Runners
#   - Group Runner : Groupe > Settings > CI/CD > Runners
#   - Specific Runner: Projet > Settings > CI/CD > Runners
# Le token ressemble a : GR1348941_aBcDeFgHiJkL

# Question 3 : Description du Runner
# Un nom pour identifier ce Runner (ex: "runner-docker-linux")

# Question 4 : Tags
# Des etiquettes pour cibler ce Runner dans vos pipelines
# Exemple : docker,linux,production
# Les tags sont TRES importants - on les verra plus tard

# Question 5 : Executor
# Le type d'environnement d'execution
# Choisissez "docker" pour commencer (le plus courant)

# Question 6 : Image Docker par default
# L'image utilisee si le job ne specifie pas d'image
# Exemple : node:18-alpine (leger et rapide)
```

Etape 3 : Ou trouver le token d'enregistrement

```
GitLab > Votre Projet > Settings (Parametres) > CI/CD > Runners
```

Vous verrez une section "Specific runners" avec :

- Un bouton "New project runner" (nouvelle methode)
- Ou un token a copier (ancienne methode)

IMPORTANT : Ne partagez JAMAIS ce token !

Il permet a n'importe qui d'enregistrer un Runner sur votre projet.

Etape 4 : Verifier que le Runner fonctionne

```
# Lister tous les Runners enregistres sur cette machine
sudo gitlab-runner list

# Exemple de sortie :
# Runtime platform      arch=amd64 os=linux
# runner-docker-linux  Executor=docker Token=abc123... URL=https://gitlab.com/

# Verifier le statut du service
sudo gitlab-runner status

# Lancer un job de test manuellement
sudo gitlab-runner run
```

Partie 4 : Le fichier config.toml explique ligne par ligne

Quand vous enregistrez un Runner, un fichier `config.toml` est cree automatiquement.

C'est le fichier de configuration principal du Runner.

Emplacement par default :

- Linux : `/etc/gitlab-runner/config.toml`
- Docker : `/etc/gitlab-runner/config.toml` (dans le conteneur)
- Windows : `C:\GitLab-Runner\config.toml`

```

# =====
# SECTION GLOBALE - Parametres qui s'appliquent a TOUS les Runners
# =====

# Nombre maximum de jobs executes en parallele sur CETTE machine
# Si vous avez 4 coeurs CPU, mettez 4 pour optimiser
concurrent = 4

# Intervalle en secondes entre chaque verification de nouveaux jobs
# 0 = utiliser la valeur par default (3 secondes)
# Plus la valeur est basse, plus le Runner reagit vite
# Mais cela genere plus de requetes vers GitLab
check_interval = 0

# Niveau de log : debug, info, warn, error, fatal, panic
# "info" est un bon compromis pour la production
# Utilisez "debug" uniquement pour diagnostiquer des problemes
log_level = "info"

# =====
# SECTION SESSION SERVER - Pour le terminal interactif (avance)
# =====
[session_server]
# Port d'ecoute pour les sessions interactives (debug en direct)
listen_address = "[::]:8093"
# Duree maximale d'une session en secondes (1800 = 30 minutes)
session_timeout = 1800

# =====
# SECTION RUNNERS - Configuration de chaque Runner enregistre
# =====

# Chaque bloc [[runners]] represente UN Runner enregistre
# Vous pouvez en avoir plusieurs sur la meme machine

[[runners]]
# Nom affiche dans l'interface GitLab
name = "runner-docker-linux"

# URL de votre instance GitLab
url = "https://gitlab.com/"

# Identifiant unique genere lors de l'enregistrement
id = 12345

# Token unique de ce Runner (genere automatiquement)
# NE JAMAIS le partager ou le commiter dans Git !
token = "aBcDeFgHiJkLmNoPqRsT"

# Limite du token (optionnel, pour les tokens a portee limitee)
token_obtained_at = "2025-01-15T10:00:00Z"
token_expires_at = "0001-01-01T00:00:00Z"

# Type d'executor utilise par ce Runner
# Valeurs possibles : docker, shell, docker+machine, kubernetes, etc.
executor = "docker"

# =====

```

Partie 5 : Les Executors - Docker, Shell, Docker Machine

L'**Executor** est la methode utilisee par le Runner pour executer vos jobs.

C'est comme choisir l'outil de travail de votre ouvrier.

Tableau comparatif des Executors

Executor	Isolation	Vitesse	Complexite	Cas d'usage
Shell	Aucune	Tres rapide	Simple	Tests rapides, scripts simples
Docker	Bonne	Rapide	Moyenne	La plupart des projets
Docker Machine	Bonne	Variable	Complexe	Auto-scaling cloud
Kubernetes	Bonne	Variable	Complexe	Grandes infrastructures
VirtualBox	Excellente	Lent	Complexe	Securite maximale

L'Executor Shell

Le Runner execute les commandes **directement sur la machine hôte**.

```
# Exemple : le Runner execute "npm test" directement sur le serveur
# Comme si vous tapiez la commande dans un terminal
test:
  # Pas besoin d'image Docker - on utilise la machine directement
  tags:
    - shell # On cible un Runner avec l'executor Shell
  script:
    - npm install # Installe les dependances sur la machine
    - npm test    # Lance les tests sur la machine
```

Avantages :

- Tres rapide (pas de demarrage de conteneur)
- Acces direct au systeme (GPU, peripheriques)

Inconvenients :

- Pas d'isolation (un job peut affecter les autres)
- La machine doit avoir tous les outils installes
- Risque de conflit entre projets

L'Executor Docker (le plus utilise)

Le Runner cree un **conteneur Docker** pour chaque job.

```
# Exemple : chaque job s'exécute dans un conteneur isolé
test:
  # L'image Docker à utiliser pour ce job
  image: node:18-alpine
  tags:
    - docker # On cible un Runner avec l'executeur Docker
  script:
    # Ces commandes s'exécutent DANS le conteneur
    - npm install
    - npm test
  # Quand le job termine, le conteneur est DETRUIT
  # L'environnement est toujours propre pour le job suivant !
```

Avantages :

- Isolation entre les jobs
- Environnement reproductible (même image = même résultat)
- Pas besoin d'installer les outils sur la machine hôte

Inconvénients :

- Un peu plus lent (démarrage du conteneur)
- Ne peut pas accéder au hardware directement

L'Executor Docker Machine

Crée des **machines virtuelles à la demande** dans le cloud (AWS, GCP, Azure).

```
Job demande --> Docker Machine crée une VM --> Job s'exécute --> VM détruite
```

C'est l'**auto-scaling** : quand il y a beaucoup de jobs, on crée plus de machines.

Quand c'est calme, les machines sont détruites pour économiser de l'argent.

Partie 6 : L'Executor Docker en détail

Les images Docker

L'image Docker est le **modèle** de l'environnement dans lequel votre job s'exécute.

```
# Vous pouvez specifier une image par job
test-node:
  image: node:18-alpine      # Image legere avec Node.js 18
  script:
    - node --version        # Affiche "v18.x.x"

test-python:
  image: python:3.11-slim    # Image legere avec Python 3.11
  script:
    - python --version      # Affiche "Python 3.11.x"

test-java:
  image: maven:3.9-eclipse-temurin-17 # Maven + Java 17
  script:
    - java --version        # Affiche "openjdk 17.x.x"
```

Les tags d'images recommandes

```
# BIEN : Utiliser une version precise pour la reproductibilite
image: node:18.19.0-alpine3.18

# ACCEPTABLE : Utiliser une version majeure
image: node:18-alpine

# DANGEREUX : "latest" peut changer a tout moment !
image: node:latest # A EVITER en production !
```

Les services dans GitLab CI

Les **services** sont des conteneurs supplementaires qui tournent **a cote** de votre job.

Typiquement, ce sont des bases de donnees ou des services d'infrastructure.

```
# Exemple : un job avec une base de donnees PostgreSQL
test-avec-bdd:
  image: node:18-alpine

  # Les services sont des conteneurs accessibles via le reseau
  services:
    # PostgreSQL 15 tourne dans un conteneur separe
    - name: postgres:15-alpine
      # Alias pour acceder au service (nom d'hote reseau)
      alias: db
      # Variables d'environnement pour configurer PostgreSQL
      variables:
        POSTGRES_DB: test_database      # Nom de la BDD
        POSTGRES_USER: test_user        # Utilisateur
        POSTGRES_PASSWORD: test_password # Mot de passe

    # Redis 7 tourne dans un autre conteneur
    - name: redis:7-alpine
      alias: cache-redis

  # Variables pour que notre application trouve la BDD
  variables:
    DATABASE_URL: "postgresql://test_user:test_password@db:5432/test_database"
    REDIS_URL: "redis://cache-redis:6379"

  # Avant le script principal, on attend que les services soient prêts
  before_script:
    # On installe un outil pour tester la connexion
    - apk add --no-cache postgresql-client
    # On attend que PostgreSQL soit prêt (health check manuel)
    - until pg_isready -h db -p 5432 -U test_user; do
        echo "En attente de PostgreSQL...";
        sleep 2;
      done
    - echo "PostgreSQL est prêt !"

  script:
    - npm install
    - npm test
```

Les volumes Docker

Les volumes permettent de **persister des données** entre les jobs ou de partager des fichiers avec le conteneur.

```
# Dans config.toml du Runner
[runners.docker]
  volumes = [
    "/cache", # Cache GitLab (obligatoire)
    "/var/run/docker.sock:/var/run/docker.sock", # Pour DinD (bind mount)
    "mon-volume:/donnees" # Volume nomme Docker
  ]
```

Partie 7 : Docker-in-Docker (DinD)

Le probleme

Vous voulez **construire une image Docker** dans votre pipeline CI/CD.

Mais votre job tourne DEJA dans un conteneur Docker !

Comment lancer Docker... dans Docker ?

Solution 1 : Docker-in-Docker (DinD)

On lance un **vrai daemon Docker** dans le conteneur du job.

```
# Pipeline avec Docker-in-Docker
build-image:
  # Image Docker officielle avec le client Docker
  image: docker:24

  # Service DinD : un daemon Docker tourne dans un conteneur a cote
  services:
    - name: docker:24-dind
      alias: docker
      # Utiliser TLS pour la securite (recommande)
      variables:
        DOCKER_TLS_CERTDIR: "/certs"

  variables:
    # Adresse du daemon Docker (le service DinD)
    DOCKER_HOST: "tcp://docker:2376"
    # Chemin vers les certificats TLS
    DOCKER_TLS_CERTDIR: "/certs"
    DOCKER_CERT_PATH: "/certs/client"
    DOCKER_TLS_VERIFY: "1"

  script:
    # Verifier que Docker fonctionne
    - docker info
    # Construire notre image
    - docker build -t mon-app:latest .
    # Lister les images
    - docker images
```

Solution 2 : Bind mount du socket Docker (plus rapide, moins securise)

```
# Cette methode utilise le Docker de la machine hote
# Plus rapide mais MOINS SECURISE car les jobs partagent le meme Docker

# Dans config.toml :
# [runners.docker]
#   volumes = ["/var/run/docker.sock:/var/run/docker.sock"]

build-image:
  image: docker:24
  # Pas besoin de service DinD !
  variables:
    DOCKER_HOST: "unix:///var/run/docker.sock"
  script:
    - docker build -t mon-app:latest .
```

Comparaison DinD vs Socket Bind

Aspect	DinD	Socket Bind
Securite	Bonne (isole)	Risqueuse (partage le Docker hote)
Vitesse	Plus lent (daemon)	Plus rapide
Cache Docker	Perdu entre les jobs	Partage avec l'hote
Configuration	Plus complexe	Simple
Production	Recommande	Acceptable en dev

Partie 8 : Build Docker multi-stage

Le concept

Un Dockerfile multi-stage utilise **plusieurs etapes** (FROM) pour :

1. **Construire** l'application avec tous les outils (compilateur, etc.)
2. **Copier** uniquement le resultat dans une image finale legere

C'est comme cuisiner dans une grande cuisine (etape build) puis servir le plat dans une assiette propre (etape finale).

Exemple concret

```
# =====
# ETAPE 1 : Construction (image lourde avec tous les outils)
# =====
# On part d'une image Node.js complete pour construire l'application
FROM node:18-alpine AS builder

# On definit le repertoire de travail dans le conteneur
WORKDIR /app

# On copie d'abord les fichiers de dependances
# Astuce : si package.json ne change pas, Docker reutilise le cache !
COPY package*.json ./

# On installe TOUTES les dependances (y compris devDependencies)
RUN npm ci

# On copie le reste du code source
COPY . .

# On construit l'application (transpilation, bundling, etc.)
RUN npm run build

# =====
# ETAPE 2 : Image finale (image legere, uniquement le necessaire)
# =====
# On repart d'une image propre et legere
FROM node:18-alpine AS production

# On definit le repertoire de travail
WORKDIR /app

# On copie UNIQUEMENT les fichiers de dependances de production
COPY package*.json ./

# On installe UNIQUEMENT les dependances de production (pas les devDeps)
RUN npm ci --only=production

# On copie UNIQUEMENT le resultat du build depuis l'etape "builder"
COPY --from=builder /app/dist ./dist

# L'utilisateur non-root pour la securite
USER node

# On expose le port de l'application
EXPOSE 3000

# On demarre l'application
CMD ["node", "dist/index.js"]
```

Gain en taille

```
Image sans multi-stage : ~900 Mo (tout le code source, devDeps, outils)
Image avec multi-stage : ~150 Mo (seulement le build + deps production)

Reduction : environ 83% !
```

Partie 9 : BuildKit - Activation et avantages

Qu'est-ce que BuildKit ?

BuildKit est le **nouveau moteur de build** de Docker. Il est plus rapide, plus intelligent et plus sécurisé que l'ancien moteur.

Avantages de BuildKit

1. **Parallelisme** : construit les étapes indépendantes en parallèle
2. **Cache intelligent** : saute les étapes qui n'ont pas changé
3. **Secrets** : passe des secrets sans les inclure dans l'image
4. **SSH forwarding** : clone des repos privés pendant le build

Activation dans GitLab CI

```
build-avec-buildkit:
  image: docker:24
  services:
    - docker:24-dind
  variables:
    # Active BuildKit pour tous les builds Docker
    DOCKER_BUILDKIT: "1"
    DOCKER_HOST: "tcp://docker:2376"
    DOCKER_TLS_CERTDIR: "/certs"
  script:
    # Le build utilise automatiquement BuildKit
    - docker build
      --build-arg BUILDKIT_INLINE_CACHE=1
      -t mon-app:latest .
```

BuildKit et le cache inline

```
build-avec-cache:
  script:
    # Etape 1 : Télécharger l'ancienne image comme source de cache
    - docker pull $CI_REGISTRY_IMAGE:latest || true

    # Etape 2 : Construire en utilisant l'ancienne image comme cache
    - docker build
      --cache-from $CI_REGISTRY_IMAGE:latest
      --build-arg BUILDKIT_INLINE_CACHE=1
      -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
      -t $CI_REGISTRY_IMAGE:latest
      .

    # Etape 3 : Pousser les nouvelles images
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
    - docker push $CI_REGISTRY_IMAGE:latest
```

Partie 10 : Cache distribue S3/GCS

Le probleme du cache local

Par default, le cache GitLab est stocke **sur le Runner** (localement).

Probleme : si vous avez **plusieurs Runners**, le cache n'est pas partage !

```
Runner 1 : a le cache de npm install (rapide)
Runner 2 : n'a PAS le cache (doit tout reinstaller, lent)
```

La solution : cache distribue

On stocke le cache dans un **stockage partage** accessible par tous les Runners.

```
Runner 1 --\
              |---> S3/GCS/MinIO (stockage partage)
Runner 2 --/

Tous les Runners lisent et ecrivent dans le meme cache !
```

Configuration S3 dans config.toml

```
# Configuration du cache distribue avec Amazon S3
[runners.cache]
# Type de stockage : "s3" pour Amazon S3 ou MinIO
Type = "s3"
# Partager le cache entre les Runners ? (true = oui)
Shared = true

[runners.cache.s3]
# Adresse du serveur S3
# Pour AWS : "s3.amazonaws.com"
# Pour MinIO local : "minio.example.com:9000"
ServerAddress = "s3.amazonaws.com"

# Cle d'accès AWS (ou MinIO)
AccessKey = "AKIAIOSFODNN7EXAMPLE"

# Cle secrete AWS (ou MinIO)
SecretKey = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"

# Nom du bucket S3 (doit exister)
BucketName = "mon-cache-gitlab-runner"

# Region AWS du bucket
BucketLocation = "eu-west-1"

# Utiliser HTTPS ? (true en production)
Insecure = false
```

Configuration GCS (Google Cloud Storage) dans config.toml

```
[runners.cache]
  Type = "gcs"
  Shared = true

[runners.cache.gcs]
  # Nom du bucket GCS
  BucketName = "mon-cache-gitlab-runner"
  # Fichier de credentials Google (JSON)
  CredentialsFile = "/etc/gitlab-runner/gcs-credentials.json"
```

Partie 11 : Strategies de cache dans .gitlab-ci.yml

Les cles de cache

La **cle de cache** determine QUAND le cache est reutilise ou renouvele.

```
# Strategie 1 : Cache par branche
# Chaque branche a son propre cache
cache:
  key: $CI_COMMIT_REF_SLUG      # Exemple : "main", "feature-login"
  paths:
    - node_modules/

# Strategie 2 : Cache par fichier (RECOMMANDEE pour npm/yarn)
# Le cache change uniquement quand package-lock.json change
cache:
  key:
    files:
      - package-lock.json      # Cle basee sur le contenu du fichier
  paths:
    - node_modules/

# Strategie 3 : Cache par branche ET fichier
cache:
  key:
    files:
      - package-lock.json
    prefix: $CI_COMMIT_REF_SLUG # Combine branche + fichier
  paths:
    - node_modules/
```

Politique de cache : pull et push

```
# Par default : pull-push (telecharge ET uploade le cache)
cache:
  key: $CI_COMMIT_REF_SLUG
  paths:
    - node_modules/
  policy: pull-push # Comportement par default

# pull : telecharge le cache mais ne le met PAS a jour
# Utile pour les jobs qui utilisent le cache sans le modifier
test:
  cache:
    key: $CI_COMMIT_REF_SLUG
    paths:
      - node_modules/
    policy: pull # Lecture seule du cache

# push : uploade le cache mais ne le telecharge PAS
# Utile pour les jobs qui creent le cache
install:
  cache:
    key: $CI_COMMIT_REF_SLUG
    paths:
      - node_modules/
    policy: push # Ecriture seule du cache
```

Cache multi-niveaux (fallback)

```
# Si le cache exact n'est pas trouve, GitLab essaie les cles de fallback
cache:
  key: ${CI_COMMIT_REF_SLUG}-${CI_COMMIT_SHA}
  paths:
    - node_modules/
  fallback_keys:
    - ${CI_COMMIT_REF_SLUG} # Fallback 1 : meme branche
    - main # Fallback 2 : branche principale
```

Partie 12 : Artifacts vs Cache - Tableau comparatif detaille

Definitions

- **Cache** : fichiers telecharges au DEBUT du job pour acclereler l'execution
- **Artifacts** : fichiers GENERES par un job et conservees APRES le job

Tableau comparatif

Aspect	Cache	Artifacts
But	Accelerer les jobs	Conserver les resultats

Quand	Telecharge au debut	Uploade a la fin
Partage	Entre jobs de meme cle	Entre stages du pipeline
Stockage	Runner ou S3/GCS	Serveur GitLab
Duree de vie	Configurable (peut expirer)	Configurable (peut expirer)
Telechargeable	Non (interne)	Oui (via l'interface GitLab)
Exemples	node_modules/, .m2/, pip cache	build/, reports/, coverage/
Fiabilite	Peut etre perdu (best-effort)	Garanti (stocke sur GitLab)
Impact reseau	Upload/download a chaque job	Upload une fois, download au besoin

Exemple concret

```

stages:
  - install      # Etape 1 : installer les dependances
  - test         # Etape 2 : lancer les tests
  - build        # Etape 3 : construire l'application

install:
  stage: install
  script:
    - npm ci
  cache:
    # On CREE le cache des node_modules
    key:
      files:
        - package-lock.json
    paths:
      - node_modules/
    policy: push                                # Ecriture seule

test:
  stage: test
  script:
    - npm test -- --coverage
  cache:
    # On UTILISE le cache des node_modules (lecture seule)
    key:
      files:
        - package-lock.json
    paths:
      - node_modules/
    policy: pull                                # Lecture seule
  artifacts:
    # On CONSERVE les rapports de tests
    paths:
      - coverage/
    reports:
      junit: junit.xml                          # Rapport de tests au format JUnit
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml

build:
  stage: build
  script:
    - npm run build
  cache:
    key:
      files:
        - package-lock.json
    paths:
      - node_modules/
    policy: pull
  artifacts:
    # On CONSERVE le build pour le deploiement
    paths:
      - dist/
    expire_in: 1 week                           # Expire apres 1 semaine

```

Partie 13 : Artifacts Reports

GitLab peut **comprendre** certains formats de rapports et les afficher directement dans l'interface.

Types de rapports

Type	Format	Affichage GitLab
junit	XML JUnit	Tests reussis/echoues dans la MR
coverage_report	Cobertura	Couverture ligne par ligne dans la MR
codequality	JSON	Problemes de qualite dans la MR
sast	JSON	Vulnerabilites de securite
dependency_scanning	JSON	Dependances vulnerables
dotenv	.env	Variables pour les jobs suivants

Rapport JUnit

```
test:
  script:
    # Lancer les tests avec un reporter JUnit
    - npm test -- --reporters=default --reporters=jest-junit
  artifacts:
    # Toujours garder les rapports meme si le job echoue
    when: always
  reports:
    # GitLab affichera les tests dans la Merge Request
    junit: junit.xml
```

Rapport de couverture de code

```
test-coverage:
  script:
    - npm test -- --coverage --coverageReporters=cobertura
  artifacts:
    when: always
  reports:
    coverage_report:
      # Format Cobertura (standard XML)
      coverage_format: cobertura
      # Chemin vers le fichier de couverture
      path: coverage/cobertura-coverage.xml
    # Expression reguliere pour extraire le pourcentage de couverture
    coverage: '/All files\s*\|s*([\d.]+)/'
```

Rapport de qualite de code

```
code-quality:
  image: docker:24
  services:
    - docker:24-dind
  script:
    # Utiliser Code Climate pour analyser la qualite
    - docker run
      --env CODECLIMATE_CODE="$PWD"
      --volume "$PWD":/code
      --volume /var/run/docker.sock:/var/run/docker.sock
      codeclimate/codeclimate analyze -f json > gl-code-quality-report.json
  artifacts:
    reports:
      codequality: gl-code-quality-report.json
```

Partie 14 : Variables CI/CD

Qu'est-ce qu'une variable CI/CD ?

Une variable CI/CD est une **valeur dynamique** accessible dans vos pipelines.

C'est comme une boîte étiquetée contenant une information.

Hierarchie des variables (ordre de priorité)

Les variables peuvent être définies à plusieurs niveaux.

Si la même variable existe à plusieurs niveaux, le niveau le plus précis l'emporte.

1. Variables de trigger (API) <-- Priorité HAUTE
2. Variables du fichier .gitlab-ci.yml (job)
3. Variables du fichier .gitlab-ci.yml (global)
4. Variables du projet (Settings > CI/CD)
5. Variables du groupe
6. Variables de l'instance GitLab <-- Priorité BASSE

Définir des variables

```
# ===== Dans le .gitlab-ci.yml =====

# Variables globales (disponibles dans TOUS les jobs)
variables:
  NODE_ENV: "test"           # Environnement Node.js
  DATABASE_HOST: "postgres"  # Hôte de la base de données

# Variables locales à un job (écrasent les globales)
test:
  variables:
    NODE_ENV: "test"          # Spécifique à ce job
    LOG_LEVEL: "debug"        # Uniquement pour ce job
  script:
    - echo $NODE_ENV          # Affiche "test"
    - echo $LOG_LEVEL          # Affiche "debug"
```

Variables protégées (protected)

Une variable PROTÉGÉE n'est disponible que sur les branches protégées (généralement "main" et les tags de release).

Pourquoi ? Pour empêcher quelqu'un de créer une branche "hack" et de lire vos secrets en ajoutant "echo \$MON_SECRET" dans un job.

Où configurer ?

Projet > Settings > CI/CD > Variables > "Protected" checkbox

Variables masquées (masked)

Une variable MASQUÉE n'apparaît pas dans les logs du job. Si le Runner affiche la valeur, elle est remplacée par [MASKED].

Exemple :

Sans masque : "Connexion à postgresql://admin:P@ssw0rd@db:5432"
Avec masque : "Connexion à postgresql://admin:[MASKED]@db:5432"

Limitations :

- La valeur doit faire au moins 8 caractères
- La valeur ne doit contenir que certains caractères
- Le masquage n'est pas parfait (base64, etc. peut révéler la valeur)

Variables de fichier (file)

```
# Une variable de type "file" cree un fichier temporaire
# contenant la valeur de la variable.
# La variable contient le CHEMIN vers ce fichier.

# Utile pour : certificats, cles SSH, fichiers de config

# Configuration dans GitLab > Settings > CI/CD > Variables :
#   Key: GOOGLE_CREDENTIALS
#   Value: (le contenu du fichier JSON)
#   Type: File

deploy:
  script:
    # $GOOGLE_CREDENTIALS contient le chemin du fichier temporaire
    # Par exemple : /tmp/CI_FILE_SPEC_abc123
    - gcloud auth activate-service-account --key-file=$GOOGLE_CREDENTIALS
```

Partie 15 : Variables predefinies GitLab

GitLab fournit automatiquement des **dizaines de variables** dans chaque job.

Les plus utilisees

```

# ===== Informations sur le commit =====
CI_COMMIT_SHA      # Hash complet du commit (ex: "a1b2c3d4e5f6...")
CI_COMMIT_SHORT_SHA # Hash court (ex: "a1b2c3d")
CI_COMMIT_BRANCH    # Nom de la branche (ex: "main", "feature-x")
CI_COMMIT_TAG        # Nom du tag (ex: "v1.0.0") - vide si pas de tag
CI_COMMIT_MESSAGE    # Message du commit
CI_COMMIT_REF_NAME   # Branche ou tag (selon le contexte)
CI_COMMIT_REF_SLUG   # Version "URL-safe" (ex: "feature/x" -> "feature-x")

# ===== Informations sur le pipeline =====
CI_PIPELINE_ID      # ID unique du pipeline (ex: 123456)
CI_PIPELINE_SOURCE   # Ce qui a declenche le pipeline : push, merge_request, schedule...
CI_PIPELINE_URL      # URL du pipeline dans GitLab

# ===== Informations sur le job =====
CI_JOB_ID           # ID unique du job
CI_JOB_NAME          # Nom du job (ex: "test", "build")
CI_JOB_STAGE         # Nom du stage (ex: "test", "deploy")
CI_JOB_TOKEN         # Token temporaire pour acceder a l'API GitLab
CI_JOB_URL           # URL du job dans GitLab

# ===== Informations sur le projet =====
CI_PROJECT_ID        # ID du projet (ex: 42)
CI_PROJECT_NAME       # Nom du projet (ex: "mon-app")
CI_PROJECT_PATH       # Chemin complet (ex: "mon-groupe/mon-app")
CI_PROJECT_URL        # URL du projet
CI_PROJECT_DIR        # Repertoire ou le code est clone

# ===== Informations sur le registry Docker =====
CI_REGISTRY          # Adresse du registry (ex: "registry.gitlab.com")
CI_REGISTRY_IMAGE     # Image du projet (ex: "registry.gitlab.com/groupe/projet")
CI_REGISTRY_USER      # Utilisateur pour se connecter au registry
CI_REGISTRY_PASSWORD  # Mot de passe pour le registry (= CI_JOB_TOKEN)

# ===== Informations sur la Merge Request =====
CI_MERGE_REQUEST_ID   # ID de la MR
CI_MERGE_REQUEST_IID   # IID de la MR (numero visible dans GitLab)
CI_MERGE_REQUEST_TITLE # Titre de la MR
CI_MERGE_REQUEST_SOURCE_BRANCH_NAME # Branche source
CI_MERGE_REQUEST_TARGET_BRANCH_NAME # Branche cible

```

Exemple d'utilisation

```

build:
  script:
    # Tagger l'image Docker avec le hash du commit
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA .

    # Se connecter au registry GitLab
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY

    # Pousser l'image
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

    # Afficher des informations utiles
    - echo "Image construite : $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA"
    - echo "Pipeline : $CI_PIPELINE_URL"
    - echo "Commit : $CI_COMMIT_MESSAGE"

```

Partie 16 : Integration Vault (concepts)

Qu'est-ce que Vault ?

HashiCorp Vault est un outil de gestion centralisée des secrets.

Au lieu de stocker vos secrets dans les variables GitLab, vous les stockez dans Vault et GitLab les récupère au moment de l'exécution.

Pourquoi utiliser Vault ?

SANS Vault :

- Secrets dupliqués dans chaque projet GitLab
- Pas de rotation automatique
- Pas d'audit (qui a accès à quel secret ?)
- Un admin GitLab peut voir tous les secrets

AVEC Vault :

- Secrets centralisés en un seul endroit
- Rotation automatique des mots de passe
- Audit complet de chaque accès
- Séparation des responsabilités (équipe sécurité vs devs)

Configuration dans .gitlab-ci.yml

```
# Déclarer les secrets à récupérer depuis Vault
deploy:
  # Les secrets sont injectés comme variables d'environnement
  secrets:
    DATABASE_PASSWORD:
      vault:
        # Chemin dans Vault : engine/data/path#field
        # Exemple : kv/data/production/database#password
        engine:
          name: kv-v2           # Moteur de secrets (Key-Value v2)
          path: production      # Chemin dans le moteur
          path: database        # Sous-chemin
          field: password       # Champ spécifique
      script:
        # $DATABASE_PASSWORD contient la valeur récupérée depuis Vault
        - echo "Connexion avec le mot de passe de Vault..."
        - ./deploy.sh
```

Architecture Vault + GitLab

```
Developpeur --> git push --> GitLab CI
      |
      | Job démarre |
      |             |
      | GitLab demande un token JWT
      |             |
      | Vault vérifie le token JWT
      |             |
      | Vault retourne le secret
      |             |
      | Le job utilise le secret
      |             |
      | Le secret expire après le job
```

Partie 17 : Bonnes pratiques et checklist

Checklist Runner

```
[ ] Utiliser l'executor Docker (sauf besoin spécifique)
[ ] Définir des tags clairs et cohérents
[ ] Limiter les ressources CPU et mémoire
[ ] Activer le monitoring Prometheus
[ ] Mettre à jour le Runner régulièrement
[ ] Utiliser des images Alpine (plus légères)
[ ] Configurer le nettoyage automatique des images Docker
[ ] Séparer les Runners de dev et de production
```

Checklist Cache

```
[ ] Utiliser des clés basées sur les fichiers de lock
[ ] Configurer la politique pull/push correctement
[ ] Utiliser un cache distribué si plusieurs Runners
[ ] Définir une durée d'expiration
[ ] Ne PAS mettre node_modules en artifact (utiliser le cache)
[ ] Configurer des fallback_keys
```

Checklist Artifacts

```
[ ] Définir expire_in pour éviter de remplir le stockage
[ ] Utiliser when: always pour les rapports de tests
[ ] Configurer les reports (junit, coverage) pour les MR
[ ] Ne PAS stocker de secrets dans les artifacts
[ ] Utiliser des chemins spécifiques (pas de wildcards larges)
```

Checklist Variables

```
[ ] Protéger les variables sensibles (Protected)
[ ] Masquer les secrets dans les logs (Masked)
[ ] Utiliser des variables de type File pour les certificats
[ ] Ne JAMAIS echo les secrets dans les scripts
[ ] Utiliser Vault pour les environnements critiques
[ ] Documenter les variables requises dans le README
```

Checklist Docker Build

```
[ ] Utiliser le multi-stage build
[ ] Activer BuildKit (DOCKER_BUILDKIT=1)
[ ] Utiliser --cache-from pour accélérer les builds
[ ] Scanner les images pour les vulnérabilités
[ ] Utiliser des images de base minimales (Alpine, distroless)
[ ] Ne PAS copier les fichiers inutiles (.dockerignore)
[ ] Fixer les versions des images de base
[ ] Exécuter en tant qu'utilisateur non-root
```

Resume du Jour 2

Ce que nous avons appris aujourd'hui :

1. RUNNERS : Les ouvriers qui executent nos pipelines
 - Shared (partages), Group (groupe), Specific (dedie)
 - Installation et enregistrement
 - Configuration via config.toml
2. EXECUTORS : Comment le Runner execute les jobs
 - Shell : directement sur la machine
 - Docker : dans un conteneur isole (recommande)
 - Docker Machine : auto-scaling cloud
3. DOCKER AVANCE : Construire des images dans le CI
 - Docker-in-Docker (DinD)
 - Multi-stage builds
 - BuildKit
4. CACHE : Accelerer les pipelines
 - Cles, politiques pull/push
 - Cache distribue S3/GCS
 - Fallback keys
5. ARTIFACTS : Conserver les resultats
 - Reports : junit, coverage, codequality
 - Difference avec le cache
6. VARIABLES : Configuration dynamique
 - Hierarchie, protected, masked
 - Variables predefinies
 - Integration Vault

Demain (Jour 3) : Environnements, deploiement et strategies avancees !

Document cree pour la formation CI/CD - Jour 2

Niveau : Debutant

Duree estimee de lecture : 2 heures